COMP1549 (2025/2026)	**Advanced Programming Coursework**	**Contribution 100% of course**
Course Leader: Dr Muhammad Taimoor Khan	**Faculty Header ID:**	**Deadline Date March 30th, 2026 23:30 UK Time**

Plagiarism is presenting somebody else's work as your own. It includes copying information directly from the Web or books without referencing the material; submitting joint coursework as an individual effort; copying another student's coursework; stealing coursework from another student and submitting it as your own work. Suspected plagiarism will be investigated and if found to have occurred will be dealt with according to the procedures set down by the University. Please see your student handbook for further details of what is/isn't plagiarism.

All material copied or amended from any source (e.g., internet, books) must be referenced correctly according to the Harvard reference style.

Your work will be submitted for plagiarism checking. Any attempt to bypass our plagiarism detection systems will be treated as a severe Assessment Offence.

Coursework Submission Requirements

- **An electronic copy of your work for this coursework must be fully uploaded by midnight on or before the Deadline Date.**
- **For this coursework you must submit a single ZIP file.**
- **Make sure that any files you upload are virus-free and NOT protected by a password or corrupted otherwise they will be treated as null submissions.**
- **Your work will not be printed in colour. Please ensure that any pages with colour are acceptable when printed in Black and White.**

Coursework Regulations

1. **If you have Extenuating Circumstances, you may submit your coursework up to two weeks after the published deadline without penalty, but this is subject to acceptance of your claim by the Faculty Extenuating Circumstances Panel.**
2. **Late submissions will be dealt with in accordance with University Regulations.**
3. **Coursework submitted more than two weeks late may be given feedback but will be recorded as a non-submission regardless of any extenuating circumstances.**
4. **Do not ask the lecturers for extensions to published deadlines - they are not authorised to award an extension.**
5. **The coursework must be submitted as above. Under no circumstances can they be accepted by academic staff.**

Please refer to the University Portal for further detail regarding the University Academic Regulations concerning Extenuating Circumstances claims.

COURSEWORK ORGANISATION

This coursework contributes to 100% of your final grade of this module. This coursework can be submitted **either individually or in a group of up to 6 students, preferably >=4 students in a group**. You must attempt ONE of the two given tasks.

The coursework must be implemented in Java and should be submitted as **a single ZIP** file that contains source code of project implementation, a **PDF** of the project self-reflection report, and a "**contributions.txt**" file that includes the percentage of contribution of each member of the group. In principle, we expect each member to contribute equally to the coursework where the sum of contribution from all members of a group equals 100%.

There will be a separate **live demonstration** of 5-10 minutes to show how key requirements of the course work has been implemented. All the group members must attend the demonstration. The demonstrations will take place during last week of term during lecture and tutorial slots of the module. The exact time for your group will be released one week before the demonstration.

The assessment of the coursework report will be based on its completeness, correctness, readability, and conformance to the expected format. While the assessment of the source code will be based on the evaluation and testing of your code against the detailed technical tasks listed in the next section of the document.

The coursework requires you to include the submission and evaluation of the following three elements:

1) project source code [**50 marks**]
2) project reflection report [**20 marks**]
3) live demonstration [**30 marks**]

TASK 1

The aim of this task is to implement a Graphical User Interface (GUI) or Command-Line Interface (CLI) based application for a group-based communication over the network following a client-server model, which conforms with the following requirements. When a new member joins (connects), he/she may provide the following parameters as an input (i.e., command-line parameters or parameters set via the GUI):

1. an ID (please ensure that each member is assigned a unique ID) of the client and
2. optionally a port and IP address of the server.

If a member is the first connection in the group, the member automatically becomes the coordinator and must be informed about it at the start-up. Later, whoever joins must be informed about the details of the current coordinator. The coordinator maintains state of active group members (e.g., their IDs) by a periodic ping (i.e., say every 20 seconds). Any member can request details of existing members from the coordinator and will receive everyone's IDs, IP addresses and ports including the current group coordinator. Based on the details, anyone can send private or broadcast messages to every other member through the server. Anyone can leave the group at any time, however, if the

coordinator leaves, then any existing member will become a coordinator. In case any member leaves, the communication among the remaining group members should not be interrupted (i.e., fault tolerant). The system should print out messages sent to/by the members.

Importantly, your implementation can either run automatically or manually. In the former case, your program may simulate the above task automatically without accepting any input parameters from users while running. For instance, a client and server may exchange different messages periodically. In the latter case, your program requires user input to simulate the task. For instance, a user will type a message from a specific client to send it to other members or server. Any member can quit by a simple ctrl-C command (CLI) or having a Quit button (GUI).

For further details about the technical background required for the task, please see lecture material, or ask the instructors.

The project implementation **must demonstrate the following programming principles and practices, which contribute directly to the final grade**:

- Using design patterns
- JUnit based testing of the application
- Fault tolerance

Assessment guidelines

Implementation (source code): The assessment of the project implementation is purely based on the correct implementation of technical requirements (as described in Task) and its adherence to various principles (as stated in Task).

Report (project reflection): The assessment of the project report is based on the overall technical quality, relevancy and completeness, contributions, and the details in supporting your solution and implementation. The report should complete one table for each of the following components considering your implementation in addition to the academic integrity statement.

- Using design patterns
- JUnit based testing of the application
- Fault tolerance

Here is the template of the table for design patterns implementation:

Design Pattern Name	Involved classes/methods	Justification
One design pattern per row	The name of the class(es) and/or methods that implement the design pattern	Why did you choose to use the mentioned design pattern?

Here is the template of the table for testing implementation:

Test Name	Involved classes/methods	Description
One test per row	The name of the class(es) and/or methods that implement the test	How does the test validate a target functionality?

Here is the template of the table for fault tolerance implementation:

Fault Tolerance Feature	Involved classes/methods	Description
One feature of fault tolerance per row, e.g., when non-coordinator leaves, when coordinator leaves	The name of the class(es) and/or methods that implement the feature	How does the implementation achieve the required feature?

All the group members must adhere to the following academic integrity statement.

"You may use (Gen)AI programs to help generate ideas. However, you should note that the material generated by these programs may be inaccurate, incomplete, or otherwise problematic. Please note that using such programs supress your own independent thinking and creativity.

You should not submit any work generated by an AI program as your own. If you include material generated by an AI program, it should be cited like any other reference material. If any core part of your submission is found to be generated by AI programs but not cited, you may fail the module.

Any plagiarism or other form of cheating will be dealt with severely under relevant University of Greenwich policies."

The following table must be submitted to avoid the AI driven plagiarism.

AI Program	Classes and/or Methods	Contribution
Name and URL of the AI program used, e.g., ChatGPT. You may include other references provided by the AI program.	The names of the class(es) and/or methods that used the AI generated contents	What percentage of the involved classes and/or methods is generated by AI program

Demonstration: Each group must demonstrate their project. Further details and time slots for the demonstration will be sent later. You should demonstrate the following two items:

A. Scenario Demonstration

1) You should run a server and three clients, where one of them is coordinator
2) Demonstrate how the coordinator works
3) Send a message from one of the clients and reply to it back (show private messaging and broadcast messaging).
4) Quit one client (NOT coordinator) and show the rest two can still communicate
5) Run another client
6) Quit coordinator now and demonstrate that new coordinator is automatically selected

B. Implementation Inspection

1) Explain the code in your implementation that implements the most challenging design pattern.
2) Explain the code in your implementation that implements the most challenging JUnit test.

For project implementation, breakdown of the marks is as follows:

- The project should demonstrate the following programming principles and practices:
  - Group formation, connection, and communication [10 marks]
    - A group should be correctly formed connecting with all members where all members can communicate without any error.
  - Group state maintenance [05 marks]
    - The state of the group must be maintained correctly. This includes recording of the messages exchanged among members of the group with timestamps.
  - Coordinator selection [05 marks]
    - A correct implementation to automatically choose the coordinator even when the existing coordinator is disrupted/disconnected abnormally.
  - Use of design patterns [10 marks]
    - Adequate use of various design patterns in the implementation of the project.
  - Fault tolerance [10 marks]
    - Adequate strategy implementation for the fault tolerance, when a member terminates abnormally or when a coordinator terminates abnormally.
  - JUnit based testing of the application [10 marks]
    - Desired testing for implementation of all of the main requirements.

For the project report, you are asked to adhere to ALL the following rules:

- The report should be written using Times New Roman font size 11.
- The paper length should not exceed **3 pages**.
- The table of content of the report should include the following tables and presentation style:
  - Design Pattern [04 marks]
    - Justification of up to 2 challenging design patterns used in your implementation.
  - JUnit Testing [04 marks]
    - Describe up to 4 challenging unit tests used in your implementation.
  - Fault Tolerance [04 marks]
    - Explain different fault tolerance scenarios implemented in your solution.
  - Usage of AI [04 marks]
    - All of the usage of AI program to generate parts of your implementation.
  - Presentation style [04 marks]
    - The presentation includes structure and contents of the report. The contents of the report should be adequate supported by reasonable justification.

For the project demonstration, you are asked to adhere to ALL the following rules:

- The demo should include the following assessments:
  - Scenario demonstration [15 marks]
    - Successful execution of the scenario as mentioned above.
  - Implementation inspection [15 marks]
    - Adequate and satisfactory explanation of the implementation as mentioned above.

- Contribution of AI
    - If most of the core requirements of the coursework are implemented using AI program, the demo will be failed.

TASK 2

The aim of this task is to implement a Java-based application to enforce security rules by design, helping prevent common programming errors that lead to security vulnerabilities. Specifically, the application will ensure that incorrect access to resources is restricted at compile time wherever possible to avoid program failures at run-time. To this end, you will develop a small security library and demo application that models a realistic access-controlled system. The application must include the following concepts:

- Users
    - With unique ID and a role can access different resources
- Roles
    - e.g. GUEST, STUDENT, STAFF, and ADMIN have different access scopes
- Resources
    - List of resources that can be accessed by different users, e.g., Printer, Exam Paper, and Lecture Material
- Access Scope
    - That establishes (non-exclusive) sensitivity of the resources, e.g.
        - PUBLIC (anyone can read),
        - INTERNAL (students/staff can read), and
        - CONFIDENTIAL (restricted access)
- Access Rules (Policy)
    - The rules to access different resources, e.g.
        - a STUDENT can read INTERNAL data but cannot access CONFIDENTIAL data, and
        - an ADMIN can read and write all resources.
- Typed Capabilities using Generics
    - That establish different (non-exclusive) capabilities of resources, e.g.,
        - Capability<Read> allows reading but cannot be used for writing,
        - Capability<Write> is required to modify data.
- Log
    - Records all the required data that can enable checking consistency of the access rules, e.g.,
        - 20-01-2025 16:00, user2, ADMIN, Exam Paper, WRITE, ALLOW
        - 21-01-2025 18:00, user2, STUDENT, Exam Paper, WRITE, REFUSE

      This information can be logged when an access to a specific resource is allowed or refused.

For further details about the technical background required for the task, please see related lecture material, or ask the instructors.

The project implementation **must demonstrate the following programming principles and practices, which contribute directly to the final grade**:

- Using Java Generics
- Using design patterns
- JUnit based testing of the application
- Enforcement of access control policy

Assessment guidelines

Implementation (source code): The assessment of the project implementation is purely based on the correct implementation of technical requirements (as described in Coursework Task) and its adherence to various principles (as stated in Coursework Task).

Report (project reflection): The assessment of the project report is based on the overall technical quality, relevancy and completeness, contributions, and the details in supporting your solution and implementation. The report should complete one table for each of the following components considering your implementation in addition to the academic integrity statement.

- Using Java Generics
- Using design patterns
- JUnit based testing of the application
- Enforcement of access control policy

Here is the template of the table for Java Generics implementation:

Design Pattern Name	Involved classes/methods	Justification
One usage of Java Generics per row	The name of the class(es) and methods that implement the Java Generics	Why did you choose to use the mentioned way of using Java Generics?

Here is the template of the table for design patterns implementation:

Design Pattern Name	Involved classes/methods	Justification
One design pattern per row	The name of the class(es) and/or methods that implement the design pattern	Why did you choose to use the mentioned design pattern?

Here is the template of the table for testing implementation:

Test Name	Involved classes/methods	Description
One test per row	The name of the class(es) and/or methods that implement the test	How does the test validate a target functionality?

Here is the template of the table for fault tolerance implementation:

Access Control Rules	Involved classes/methods	Description
One access control rule per row	The name of the class(es) and/or methods that implement the rule	How does the implementation enforce the mentioned rule?

All the group members must adhere to the following academic integrity statement.

"You may use (Gen)AI programs to help generate ideas. However, you should note that the material generated by these programs may be inaccurate, incomplete, or otherwise problematic. Please note that using such programs supress your own independent thinking and creativity.

You should not submit any work generated by an AI program as your own. If you include material generated by an AI program, it should be cited like any other reference material. If any core part of your submission is found to be generated by AI programs but not cited, you may fail the module.

Any plagiarism or other form of cheating will be dealt with severely under relevant University of Greenwich policies."

The following table must be submitted to avoid the AI driven plagiarism.

AI Program	Classes and/or Methods	Contribution
Name and URL of the AI program used, e.g., ChatGPT. You may include other references provided by the AI program.	The names of the class(es) and/or methods that used the AI generated contents	What percentage of the involved classes and/or methods is generated by AI program

Demonstration: Each group must demonstrate their project. Further details and time slots for the demonstration will be sent later. You should demonstrate the following two items:

C. Scenario Demonstration

1) You should run the application with at least 3 different users who have different access to different resources
2) Demonstrate how each user is successfully allowed to access the resources following the access rules
3) Demonstrate how each user is denied access to the resources following the access rules
4) Show the logged messages that also confirm allowed and denied access of the above two cases

D. Implementation Inspection

3) Explain the code in your implementation that implements the most challenging design pattern.
4) Explain the code in your implementation that implements the most challenging JUnit test.

For project implementation, breakdown of the marks is as follows:

- The project should demonstrate the following programming principles and practices:
  - Concepts implementation [06 marks]
    - Including users, their roles, and resources.
  - Access rules implementation [06 marks]
    - The policy rules that enforce access by allowing specific users or refusing them
  - Scope [06 marks]
    - Different scopes are implemented.
  - Capabilities [06 marks]
    - Different capabilities are implemented that support different access operations on resources by different users
  - Log [06 marks]
    - The enforcement of the access rules must be maintained correctly. This includes recording of the messages with correct details.
  - Use of Java Generics [08 marks]

    - Adequate use of Java Generics in the implementation of the project.
- Use of design patterns [06 marks]
    - Adequate use of various design patterns in the implementation of the project.
- JUnit based testing of the application [06 marks]
    - Desired testing for implementation of all the main requirements.

For the project report, you are asked to adhere to ALL the following rules:

- The report should be written using Times New Roman font size 11.
- The paper length should not exceed **3 pages**.
- The table of content of the report should include the following tables and presentation style:
    - Java Generics [04 marks]
        - Justification of up to 2 challenging uses of Java Generics in your implementation.
    - Design Pattern [04 marks]
        - Justification of up to 2 challenging design patterns used in your implementation.
    - JUnit Testing [04 marks]
        - Describe up to 4 challenging unit tests used in your implementation.
    - Access Control Policy [04 marks]
        - Describe up to 4 challenging access rules implemented in your solution.
    - Usage of AI [02 marks]
        - All the usage of AI program to generate parts of your implementation.
    - Presentation style [02 marks]
        - The presentation includes structure and contents of the report. The contents of the report should be adequate supported by reasonable justification.

For the project demonstration, you are asked to adhere to ALL the following rules:

- The demo should include the following assessments:
    - Scenario demonstration [15 marks]
        - Successful execution of the scenario as mentioned above.
    - Implementation inspection [15 marks]
        - Adequate and satisfactory explanation of the implementation as mentioned above.
    - Contribution of AI
        - If most of the core requirements of the coursework are implemented using AI program, the demo will be failed.

Note: There is a 20% penalty if the paper does not abide by all above rules. Furthermore, first the submission will be evaluated based on the above criteria. Later, marks for each group member will be calculated based on their submitted percentage of contribution. For instance, if a group of 5 students with equal contribution (i.e., 20% each) achieves 80%, then a member of the group who has 20% contribution will get full 80% but if someone had 10% contribution, he would get only 40% score.